Poneglyph API Documentation

REST API of a community based Q&A platform: communities, posts, comments, reactions, moderation and administration.

Version 1.0 | 11 October 2026
90 endpoints | Node.js, Express 5, Prisma, PostgreSQL

Contents

1. Introduction . . . 2
1.1 Conventions used in this document . . . 2

2. Authentication and roles . . . 2
2.1 Roles . . . 2

3. Common conventions . . . 3
3.1 Errors . . . 3
3.2 Pagination . . . 3
3.3 Sorting . . . 4
3.4 Filtering posts . . . 4
3.5 Uploads . . . 4
3.6 Statuses and visibility . . . 4

4. Quick start . . . 5

5. Endpoint reference . . . 6
5.1 Authentication (/api/auth) . . . 6
5.2 Users (/api/users) . . . 6
5.3 Communities (/api/communities) . . . 7
5.4 Local categories of a community . . . 8
5.5 Posts (/api/communities/:community_id/posts) . . . 9
5.6 Comments (/api/communities/:community_id/posts/:post_id/comments) . . . 12
5.7 Global categories (/api/global-categories) . . . 12
5.8 Feeds (/api/posts) . . . 12
5.9 Reports . . . 13
5.10 Category requests . . . 15
5.11 Community moderation panel (/api/communities/:community_id/moderation) . . . 16
5.12 Site admin panel (/api/admin) . . . 19

6. Main objects . . . 23
6.1 Post . . . 23
6.2 Community . . . 24
6.3 User . . . 24
6.4 Report . . . 25

7. Rules and permissions in short . . . 26

8. Error examples . . . 26

Appendix A. Enumerations . . . 27

Appendix B. Activity log actions . . . 28

1. Introduction

Poneglyph is a Reddit-style platform for programmers: a forum of forums. Users create **communities**; every community has an owner and moderators, posts live inside communities and are organised by the community's own **local categories**, while communities themselves are grouped by **global categories**. This document describes the HTTP API that every client (the web panel, the future frontend, scripts, tests) uses.

Item	Value
Base URL	`http://localhost:5000/api` (the port comes from the PORT variable)
Format	JSON requests and responses (`Content-Type: application/json`); file uploads use `multipart/form-data`
Authentication	Bearer token in the `Authorization` header, obtained from `POST /api/auth/login`
Uploaded files	served from `/storage/<file name>` (the URL is stored in `media_url` and `avatar_path`)
Admin panel	a web interface for the moderation endpoints, served at `/admin/`
Identifiers	positive integers; an invalid id in a path answers 400
Dates	ISO 8601 strings in UTC, for example `2026-10-11T18:05:04.000Z`

1.1 Conventions used in this document

- **Access** column: *public* (no token), *optional* (works without a token and shows more to a signed-in viewer), *user* (any signed-in user), *member*, *author*, *staff* (community owner or moderator, or an admin), *owner*, *site staff* (admin or site moderator), *admin*.
- Request bodies are shown as JSON; a field written as *optional* may be omitted.
- Examples marked **real** were produced by calling a running server with the demo data, with long values shortened.

2. Authentication and roles

A user registers, confirms the email address with the link that is mailed to them, and then logs in. Login answers with a token that must be sent in every protected request:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

- A token lives up to one day (configurable). **Logout**, a password change, a password reset, a role change and an account suspension invalidate every token of the user at once.
- Verification and reset tokens exist only in the emails: the API never returns them.
- Rate limits per IP address and 15 minutes: login 10, registration 20, password reset and resend of the verification email 5. Exceeding them answers 429.
- A **suspended** account cannot log in; with an old token it gets 401, with a still valid token 403 on protected routes.

2.1 Roles

Role	Kind	What it is
USER	site	the default role of every account
MODERATOR	site	a cut-down admin: sees all reports, hides and restores content; cannot delete, ban, suspend, see emails or manage categories
ADMIN	site	can do everything; hands out roles, suspends accounts, decides about global categories
OWNER	community	creator of a community; manages it, appoints moderators, transfers ownership
MODERATOR	community	moderates one community: handles reports, bans, hides content, manages local categories
MEMBER	community	joined the community and may post in it

The same word "moderator" names two different things: the site role is stored on the account, the community role on the membership. A site moderator has no powers inside a community beyond hiding and restoring content.

3. Common conventions

3.1 Errors

Every error has the same shape. The text is meant to be shown to a person.

```
{ "message": "Human readable explanation" }
```

Status	Meaning
200 / 201	success / created (post, comment, community, member, reaction, report, request, ban, user)
400	invalid input, an invalid id or query parameter, a request without a body, a wrong file type
401	no token, an invalid or expired token, or a token invalidated by logout, role change or suspension
403	signed in but not allowed (also: banned from the community, suspended account)
404	does not exist, **or is hidden from this viewer**: hidden content is indistinguishable from missing content
409	conflict: duplicate login, email or title; a report or request that is already handled; deleting the default category, the last admin or an owner
413	the request body is too large
429	too many attempts, try again later
503	the verification email could not be sent
500	unexpected error; details are never exposed

3.2 Pagination

Lists answer with a **meta** object and the items in **data**. Use **page** (default 1) and **limit** (default 10, at most 100).

```
{ "meta": { "total": 9, "page": 1, "limit": 2, "totalPages": 5 }, "data": [ ... ] }
```

Category lists, the comment tree of a post and a few small lists are returned in full or by top-level entries; each such endpoint says so.

3.3 Sorting

Lists accept **sort** and **order** (**asc** or **desc**). An unknown value answers 400 with the list of allowed values. Defaults are in the second column.

List	sort values	Default
Posts	likes, date, comments, title	likes, descending
Comments	likes, date	likes, descending (date: oldest first)
Users	rating, date, login, name	rating, descending
Communities	members, posts, date, title	members, descending
Community members	joined, login, rating	joined, oldest first
Global categories	title, communities	title, ascending
Local categories	title, posts (the default category always comes first)	title, ascending
Reports (panels)	date, reason, status	date, newest first
Bans (panel)	date, login, expires	date, newest first
Category requests	date, title, status	date, newest first
Activity log	date, action	date, newest first
Site users (admin panel)	rating, date, login, name, posts	rating, descending

3.4 Filtering posts

Parameter	Values	Meaning
`categories`	1,2,3	posts that have any of these local categories
`date_from, date_to`	ISO dates	publication interval
`status`	ACTIVE, INACTIVE, DRAFT, ARCHIVED	only among posts the viewer may see
`author_id`	user id	posts of one author
`search`	text	case-insensitive search in title and content

3.5 Uploads

- Send **multipart/form-data**. Posts take up to 8 files in the field **media**, comments up to 5, avatars one file in the field **avatar**.
- Allowed types: JPEG, PNG, WebP, GIF; videos (MP4, WebM, OGG, QuickTime) only in posts and comments. Avatars: 10 MB, post and comment media: 100 MB.
- A request that fails validation leaves no uploaded files behind.

3.6 Statuses and visibility

Status	Who sees it
ACTIVE	everyone
INACTIVE	hidden by moderation: the author, the community staff and site staff; everybody else gets 404
DRAFT (posts)	only the author
ARCHIVED (posts)	the author and the staff

- A comment hidden by a moderator hides its whole reply thread.
- Comments and reactions are accepted only under **active** posts and on **active** comments.
- Only the author edits the text and media of a post or comment, nobody else, not even an admin. Moderators change the **status** and move posts between categories.

- An author cannot undo a moderator's decision: a post set to INACTIVE by staff can only be restored by staff.

4. Quick start

The shortest path from nothing to a published post:

- **Register:** `POST /api/auth/register` with login, email, full_name, password and password_confirmation. A verification email is sent.
- **Confirm the email:** open the link from the email, which calls `GET /api/auth/verify?token=...`.
- **Log in:** `POST /api/auth/login` and keep the returned *token*.
- **Find or create a community:** `GET /api/communities` or `POST /api/communities`; join with `POST /api/communities/:community_id/join`.
- **Post:** `POST /api/communities/:community_id/posts` with title and content (the post goes to the default category *General* if you give none).
- **Read, react, discuss:** list posts, `POST .../like`, `POST .../comments`.

Step: log in (real)

Request

```
POST /api/auth/login
Content-Type: application/json

{
  "login": "bob",
  "password": "Password123!"
}
```

Response

```
HTTP 200
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsIn...",
  "user": {
    "id": 3,
    "login": "bob",
    "full_name": "Bob Smith",
    "avatar_path": "default_avatar.png",
    "rating": 9,
    "role": "USER",
    "created_at": "2026-10-11T19:18:59.441Z",
    "email": "bob@example.com",
    "is_email_confirmed": true
  }
}
```

5. Endpoint reference

All paths start with `/api`. `:community_id` and similar segments are numeric ids. Total: **90 endpoints**.

5.1 Authentication (/api/auth)

Method	Path	Access	Description
POST	`/auth/register`	public	Body: **login** (3-30 letters, digits, . - _), **email**, **full_name**, **password** (8-72 characters), **password_confirmation**. Sends the verification email; if it cannot be sent the account is not created (503).
GET	`/auth/verify?token=`	public	Confirms the email address. The token is single use and valid for 24 hours.
POST	`/auth/resend-verification`	public	Body: **email**. Always answers the same message, whether the account exists or not.
POST	`/auth/login`	public	Body: **login** (login or email), **password**. Returns **token** and **user**. Requires a confirmed email; a suspended account gets 403.
POST	`/auth/logout`	user	Invalidates every token of the user.
POST	`/auth/reset-password-request`	public	Body: **email**. Sends a reset link valid for one hour; answers identically for unknown emails.
POST	`/auth/reset-password`	public	Body: **token** (from the email), **password**, optional **password_confirmation**. Single use; ends all sessions.
POST	`/auth/change-password`	user	Body: **current_password**, **new_password**, **new_password_confirmation**. Returns a fresh **token**.

5.2 Users (/api/users)

Method	Path	Access	Description
GET	`/users`	optional	Query: page, limit, search (login or name), sort. Admins also see emails.
POST	`/users`	admin	Creates a confirmed user. Body: login, email, full_name, password, password_confirmation, optional role.
PATCH	`/users/avatar`	user	multipart field **avatar**. Replaces the previous picture.
GET	`/users/:user_id`	optional	Public profile with **rating**. The email is shown only to the owner and admins.
GET	`/users/:user_id/communities`	public	Communities of the user with their **member_role**. Query: page, limit.
PATCH	`/users/:user_id`	owner / admin	Body: any of login, email, full_name; **role** (USER, MODERATOR, ADMIN) for admins only. A new email must be confirmed again.
DELETE	`/users/:user_id`	owner / admin	Deletes the account and its content. Refused (409) while the user owns a community and for the last admin.
POST	`/users/:user_id/report`	user	Reports a user. Body: **reason**, optional **details**, optional **community_id**. See section 5.9.

5.3 Communities (/api/communities)

Method	Path	Access	Description
GET	`/communities`	optional	Query: page, limit, search, sort. Every item has **viewer_role** (OWNER, MODERATOR, MEMBER or null).
POST	`/communities`	user	Body: **title** (unique), optional **description**, optional **global_category_ids**. Creates the default category *General*; the creator becomes the owner.
GET	`/communities/:community_id`	optional	Details with the local categories and global categories.
PATCH	`/communities/:community_id`	staff	Body: title, description.
PATCH	`/communities/:community_id/avatar`	staff	multipart field **avatar**.
DELETE	`/communities/:community_id`	owner / admin	Deletes the community with posts, comments, files and its moderation data.
POST	`/communities/:community_id/join`	user	Joins as MEMBER. Refused (403) for a banned user.
DELETE	`/communities/:community_id/leave`	user	Leaves. The owner cannot leave: transfer ownership first.
GET	`/communities/:community_id /members`	public	Query: page, limit, role, sort.
PATCH	`/communities/:community_id /members/:user_id`	owner / admin	Body: **role** (MODERATOR or MEMBER). The owner's role cannot be changed.
DELETE	`/communities/:community_id /members/:user_id`	staff	Removes a member. Only the owner or an admin can remove a moderator.
POST	`/communities/:community_id /transfer-ownership`	owner / admin	Body: **user_id** of a member. The previous owner becomes a moderator.
PUT	`/communities/:community_id /global-categories`	owner / admin	Body: **global_category_ids** (array). Replaces the whole set.
POST	`/communities/:community_id /category-requests`	member	Asks for a local category. Body: title, optional description. See section 5.10.

Example: list communities, signed in as a member (real)

Request

```
GET /api/communities?limit=1&sort=members
```

Response

```
HTTP 200
{
  "meta": {
    "total": 5,
    "page": 1,
    "limit": 1,
    "totalPages": 5
  },
  "data": [
    {
      "id": 5,
      "title": "Home Lab",
      "description": "Servers, networking and self-hosting",
      "avatar_path": "default_community.png",
      "created_at": "2026-10-11T19:18:59.466Z",
      "owner_id": 6,
      "owner": {
        "id": 6,
        "login": "erin",
        "full_name": "Erin Brown",
        "avatar_path": "default_avatar.png",
        "rating": 3
      },
      "global_categories": [
        {
          "id": 4,
          "title": "Hardware",
          "description": "Computers, electronics and gadgets"
        }
      ],
      "_count": {
        "members": 5,
        "posts": 2
      },
      "viewer_role": "MEMBER"
    }
  ]
}
```

5.4 Local categories of a community

Local categories organise the posts inside one community. The category **General** is created with the community, is flagged *is_default* and cannot be deleted. Lists here are returned as plain arrays, not paginated.

Method	Path	Access	Description
GET	`/communities/:community_id /categories`	public	Query: sort (title, posts), order. The default category is always first.
POST	`/communities/:community_id /categories`	staff	Body: title (up to 50 characters, unique in the community), optional description.
GET	`/communities/:community_id /categories/:category_id`	public	One category with the number of posts.
PATCH	`/communities/:community_id /categories/:category_id`	staff	Body: title, description.
DELETE	`/communities/:community_id /categories/:category_id`	staff	The default category answers 409. Posts left without a category move to the default one.
GET	`/communities/:community_id /categories/:category_id/posts`	optional	Posts of one category; same query parameters as the post list.

5.5 Posts (/api/communities/:community_id/posts)

Method	Path	Access	Description
GET	`/communities/:community_id/posts`	optional	Query: page, limit, sort, order, categories, date_from, date_to, status, author_id, search (section 3).
POST	`/communities/:community_id/posts`	member	JSON or multipart. Body: **title**, **content**, optional **categories** (ids, default: General), optional **status** (ACTIVE or DRAFT); files in **media** (max 8). Refused for banned users.
GET	`/communities/:community_id/posts /:post_id`	optional	One post. A hidden post answers 404.
PATCH	`/communities/:community_id/posts /:post_id`	author / staff	Author: title, content, categories, status (ACTIVE, DRAFT, ARCHIVED, not when INACTIVE), **removed_media_ids**, new files. Staff (and site moderators): **status** ACTIVE or INACTIVE and **categories** only.
DELETE	`/communities/:community_id/posts /:post_id`	author / staff	Deletes the post with its comments and files. Site moderators cannot delete.
GET	`/communities/:community_id/posts /:post_id/categories`	optional	Categories of the post.
GET	`/communities/:community_id/posts /:post_id/likes`	optional	likes_count, dislikes_count, score and the lists.
POST	`/communities/:community_id/posts /:post_id/like`	user	Body: **type** LIKE or DISLIKE (default LIKE). One reaction per user: repeating answers 200, switching type updates it. Only active posts.
DELETE	`/communities/:community_id/posts /:post_id/like`	user	Removes your reaction (404 if there is none).
POST	`/communities/:community_id/posts /:post_id/report`	user	Reports the post. Body: reason, optional details.

Example: the post list, sorted by likes (real)

Request

```
GET /api/posts?limit=2&sort=likes
```

Response


```
HTTP 200
{
  "meta": {
    "total": 9,
    "page": 1,
    "limit": 2,
    "totalPages": 5
  },
  "data": [
    {
      "id": 7,
      "title": "Pathfinding in a 2D game",
      "content": "Detailed question or discussion body for \"Pathfinding in a 2D ga...",
      "publish_date": "2026-10-11T19:18:59.496Z",
      "status": "ACTIVE",
      "score": 2,
      "author_id": 1,
      "community_id": 4,
      "author": {
        "id": 1,
        "login": "admin",
        "full_name": "Site Administrator",
        "avatar_path": "default_avatar.png",
        "rating": 8
      },
      "community": {
        "id": 4,
        "title": "Game Dev",
        "avatar_path": "default_community.png"
      },
      "categories": [
        {
          "id": 11,
          "title": "Questions",
          "description": null,
          "is_default": false,
          "community_id": 4
        }
      ],
      "media": [],
      "comments_count": 2,
      "viewer_reaction": null
    }
  ]
}
```

Example: create a post (real)

Request

```
POST /api/communities/1/posts
Content-Type: application/json

{
  "title": "How do I debounce a function?",
  "content": "I need to limit how often a handler runs. What is the idiomatic way?"
}
```

Response

```
HTTP 201
{
  "id": 11,
  "title": "How do I debounce a function?",
  "content": "I need to limit how often a handler runs. What is the idiomatic ...",
  "publish_date": "2026-10-11T19:19:07.285Z",
  "status": "ACTIVE",
  "score": 0,
  "author_id": 3,
  "community_id": 1,
  "author": {
    "id": 3,
    "login": "bob",
    "full_name": "Bob Smith",
    "avatar_path": "default_avatar.png",
    "rating": 9
  },
  "community": {
    "id": 1,
    "title": "JavaScript Corner",
    "avatar_path": "default_community.png"
  },
  "categories": [
    {
      "id": 1,
      "title": "General",
      "description": null,
      "is_default": true,
      "community_id": 1
    }
  ],
  "media": [],
  "comments_count": 0,
  "viewer_reaction": null
}
```

Example: like a post (real)

Request

```
POST /api/communities/1/posts/2/like
Content-Type: application/json

{
  "type": "LIKE"
}
```

Response

```
HTTP 201
{
  "id": 27,
  "type": "LIKE",
  "created_at": "2026-10-11T19:19:07.299Z",
  "author_id": 6,
  "post_id": 2
}
```

5.6 Comments (/api/communities/:community_id/posts/:post_id/comments)

Method	Path	Access	Description
GET	/comments	optional	The thread of a post: top-level comments with their **replies** nested. Query: page, limit (apply to top-level comments), sort (likes, date), order.
POST	/comments	user	Body: **content**, optional **parent_id** (a reply); files in **media** (max 5). Only under active posts; refused for banned users.
GET	/comments/:comment_id	optional	One comment.
PATCH	/comments/:comment_id	author / staff	**content**: the author only (not while the comment is locked). **status** ACTIVE or INACTIVE: community staff, admins and site moderators.
DELETE	/comments/:comment_id	author / staff	Deletes the comment with its replies. Site moderators cannot delete.
GET	/comments/:comment_id/likes	optional	Reactions of the comment.
POST	/comments/:comment_id/like	user	Body: type LIKE or DISLIKE.
DELETE	/comments/:comment_id/like	user	Removes your reaction.
POST	/comments/:comment_id/report	user	Reports the comment. Body: reason, optional details.

In this table /comments stands for the full path /api/communities/:community_id/posts/:post_id/comments.

5.7 Global categories (/api/global-categories)

Method	Path	Access	Description
GET	/global-categories	public	Plain array with the number of communities. Query: sort (title, communities), order.
POST	/global-categories	admin	Body: title (unique, up to 50 characters), optional description.
GET	/global-categories /:global_category_id	public	One category.
PATCH	/global-categories /:global_category_id	admin	Body: title, description.
DELETE	/global-categories /:global_category_id	admin	Deletes the category and its links to communities.
GET	/global-categories /:global_category_id/communities	optional	Communities in the category. Query: page, limit, sort.

5.8 Feeds (/api/posts)

Method	Path	Access	Description
GET	/posts	optional	Posts of all communities. Same query parameters as the community post list.
GET	/posts/feed	user	Posts of the communities the user has joined.

5.9 Reports

Any signed-in user can report content or a person. The report goes to the moderation queue of the community (or to the site panel when it is not tied to a community). Rules: the target must be visible to the reporter, you cannot report yourself, and one open report per target and reporter is allowed (409 otherwise).

Method	Path	Access	Description
POST	`/communities/:community_id/posts/:post_id/report`	user	Body: **reason**, optional **details** (up to 1000 characters).
POST	`/communities/:community_id/posts/:post_id/comments/:comment_id/report`	user	Same body.
POST	`/users/:user_id/report`	user	Same body plus optional **community_id** (behaviour inside a community) or none (the account in general).

Reasons: `SPAM`, `HARASSMENT`, `HATE_SPEECH`, `MISINFORMATION`, `INAPPROPRIATE`, `OTHER`. Report status: OPEN, RESOLVED, DISMISSED.

Example: report a post (real)

Request

```
POST /api/communities/1/posts/2/report
Content-Type: application/json

{
  "reason": "SPAM",
  "details": "Looks like advertising"
}
```

Response

```
HTTP 201
{
  "id": 7,
  "target_type": "POST",
  "reason": "SPAM",
  "details": "Looks like advertising",
  "target_excerpt": "Promise.all vs allSettled",
  "status": "OPEN",
  "action_taken": null,
  "resolution_note": null,
  "created_at": "2026-10-11T19:19:07.316Z",
  "resolved_at": null,
  "reporter_id": 6,
  "community_id": 1,
  "post_id": 2,
  "comment_id": null,
  "target_user_id": null,
  "resolved_by_id": null,
  "reporter": {
    "id": 6,
    "login": "erin",
    "full_name": "Erin Brown"
  },
  "community": {
    "id": 1,
    "title": "JavaScript Corner",
    "avatar_path": "default_community.png"
  },
  "post": {
    "id": 2,
    "title": "Promise.all vs allSettled",
    "status": "ACTIVE",
    "author_id": 1
  },
  "comment": null,
  "target_user": null,
  "resolved_by": null
}
```

5.10 Category requests

Members ask for new local categories; owners and moderators ask for new global ones. Local requests are decided by the community staff, global requests by admins. A request is refused if the category already exists or an identical request is pending.

Method	Path	Access	Description
POST	`/communities/:community_id /category-requests`	member	Body: **title** (up to 50 characters), optional **description**. The staff create categories directly instead.
POST	`/category-requests/global`	owner / moderator	Body: **community_id**, **title**, optional description. On approval the category is attached to that community. Admins create categories directly.
GET	`/category-requests/mine`	user	Your requests with their status. Query: page, limit, status, sort.
DELETE	`/category-requests/:request_id`	user	Cancels your own pending request.

Example: ask for a local category (real)

Request

```
POST /api/communities/1/category-requests
Content-Type: application/json

{
  "title": "Debugging tips",
  "description": "Tricks for finding bugs faster"
}
```

Response

```
HTTP 201
{
  "id": 7,
  "kind": "LOCAL",
  "title": "Debugging tips",
  "description": "Tricks for finding bugs faster",
  "status": "PENDING",
  "review_note": null,
  "created_at": "2026-10-11T19:19:07.437Z",
  "reviewed_at": null,
  "requester_id": 3,
  "community_id": 1,
  "reviewed_by_id": null,
  "requester": {
    "id": 3,
    "login": "bob",
    "full_name": "Bob Smith"
  },
  "community": {
    "id": 1,
    "title": "JavaScript Corner",
    "avatar_path": "default_community.png"
  },
  "reviewed_by": null
}
```

5.11 Community moderation panel (/api/communities/:community_id/moderation)

For the community owner, its moderators and admins; everybody else gets 403. Site moderators use the site panel instead.

Method	Path	Access	Description
GET	/moderation/overview	staff	Counters (members, open reports, pending category requests, active bans, posts by status, hidden comments) and the recent activity.
GET	/moderation/reports	staff	The report queue. Query: **status** (default OPEN, or ALL), **target_type** (POST, COMMENT, USER), **reason**, sort, order, page, limit. Every item has **open_reports_on_target**.
GET	/moderation/reports/:report_id	staff	One report of this community.
POST	/moderation/reports/:report_id /resolve	staff	Body: **action**, optional **note**, optional **ban_days**. Actions: NONE, HIDE_CONTENT, DELETE_CONTENT, BAN_USER. Closes the other open reports about the same target.
POST	/moderation/reports/:report_id /dismiss	staff	Body: optional **note**. The content stays as it is.
GET	/moderation/bans	staff	Active bans. Query: sort (date, login, expires), order, include_expired=true.
POST	/moderation/bans	staff	Body: **user_id**, optional **reason**, optional **duration_days** (empty = permanent). Removes the user from the community. Staff and admins cannot be banned.
DELETE	/moderation/bans/:user_id	staff	Lifts a ban.
GET	/moderation/log	staff	The activity feed. Query: action (for example post.hidden), actor_id, sort, order.
GET	/moderation/category-requests	staff	Requests for local categories. Query: status (default PENDING), sort.
POST	/moderation/category-requests /:request_id/approve	staff	Body: optional note. Creates the category.
POST	/moderation/category-requests /:request_id/reject	staff	Body: optional note.

In this table /moderation stands for /api/communities/:community_id/moderation.

Example: the overview of a community panel (real)

Request

```
GET /api/communities/1/moderation/overview
```

Response

```
HTTP 200
{
  "community_id": 1,
  "counts": {
    "members": 5,
    "open_reports": 4,
    "pending_category_requests": 1,
    "active_bans": 1,
    "posts": {
      "ACTIVE": 3,
      "INACTIVE": 0,
      "DRAFT": 0,
      "ARCHIVED": 0
    },
    "hidden_comments": 0
  },
  "recent_actions": [
    {
      "id": 1,
      "action": "user.banned",
      "target_type": "USER",
      "target_id": 8,
      "note": "Harassing other members",
      "created_at": "2026-10-11T19:18:59.887Z",
      "actor_id": 2,
      "community_id": 1,
      "actor": {
        "id": 2,
        "login": "alice",
        "full_name": "Alice Johnson",
        "role": "USER"
      },
      "community": {
        "id": 1,
        "title": "JavaScript Corner",
        "avatar_path": "default_community.png"
      }
    }
  ]
}
```

Example: the report queue (real)

Request

```
GET /api/communities/1/moderation/reports?status=OPEN&limit=1
```

Response

```
HTTP 200
{
  "meta": {
    "total": 4,
    "page": 1,
    "limit": 1,
    "totalPages": 4
  },
  "data": [
    {
      "id": 7,
      "target_type": "POST",
      "reason": "SPAM",
      "details": "Looks like advertising",
      "target_excerpt": "Promise.all vs allSettled",
      "status": "OPEN",
      "action_taken": null,
      "resolution_note": null,
      "created_at": "2026-10-11T19:19:07.316Z",
      "resolved_at": null,
      "reporter_id": 6,
      "community_id": 1,
      "post_id": 2,
      "comment_id": null,
      "target_user_id": null,
      "resolved_by_id": null,
      "reporter": {
        "id": 6,
        "login": "erin",
        "full_name": "Erin Brown"
      },
      "community": {
        "id": 1,
        "title": "JavaScript Corner",
        "avatar_path": "default_community.png"
      },
      "post": {
        "id": 2,
        "title": "Promise.all vs allSettled",
        "status": "ACTIVE",
        "author_id": 1
      },
      "comment": null,
      "target_user": null,
      "resolved_by": null,
      "open_reports_on_target": 1
    }
  ]
}
```

5.12 Site admin panel (/api/admin)

For admins and site moderators. Rows marked **admin** are refused to site moderators (403). A site moderator cannot see emails, delete content, ban, suspend or decide about global categories.

Method	Path	Access	Description
GET	`/admin/overview`	site staff	Site-wide counters, the biggest communities, the last 7 days and recent activity. The global category queue is shown to admins only.
GET	`/admin/users`	site staff	Query: search, role, suspended (true or false), sort (rating, date, login, name, posts), order. Emails only for admins; each user has counts of posts, comments and reports against them.
POST	`/admin/users/:user_id/suspend`	admin	Body: optional **reason**. Signs the user out and blocks login. Admins and yourself cannot be suspended.
POST	`/admin/users/:user_id/unsuspend`	admin	Restores the account.
GET	`/admin/reports`	site staff	Reports of every community; same filters as the community queue plus **community_id**.
GET	`/admin/reports/:report_id`	site staff	One report.
POST	`/admin/reports/:report_id/resolve`	site staff	Site moderators: NONE, HIDE_CONTENT. Admins also: DELETE_CONTENT, BAN_USER, SUSPEND_USER.
POST	`/admin/reports/:report_id/dismiss`	site staff	Body: optional note.
GET	`/admin/category-requests`	admin	Query: kind (GLOBAL, LOCAL), status (default PENDING), community_id, sort.
POST	`/admin/category-requests /:request_id/approve`	admin	Body: optional note. A global request creates the category and attaches the community.
POST	`/admin/category-requests /:request_id/reject`	admin	Body: optional note.
GET	`/admin/moderation-log`	site staff	The activity feed of the whole site. Query: community_id, action, actor_id, sort, order.

Example: the site overview (real)

Request

```
GET /api/admin/overview
```

Response

```
HTTP 200
{
  "viewer_role": "ADMIN",
  "counts": {
    "users": 9,
    "suspended_users": 0,
    "communities": 5,
    "posts": {
      "ACTIVE": 10,
      "INACTIVE": 1,
      "DRAFT": 0,
      "ARCHIVED": 0
    },
    "comments": 16,
    "hidden_comments": 0,
    "open_reports": 5,
    "pending_local_category_requests": 4,
    "pending_global_category_requests": 1,
    "active_bans": 5
  },
  "last_7_days": {
    "new_users": 9,
    "new_posts": 11
  },
  "top_communities": [
    {
      "id": 1,
      "title": "JavaScript Corner",
      "members": 5,
      "posts": 3
    }
  ],
  "recent_actions": [
    {
      "id": 8,
      "action": "category_request.rejected",
      "target_type": "CATEGORY_REQUEST",
      "target_id": 4,
      "note": "Memes | Off topic for this community",
      "created_at": "2026-10-11T19:18:59.894Z",
      "actor_id": 4,
      "community_id": 3,
      "actor": {
        "id": 4,
        "login": "carol",
        "full_name": "Carol Davis",
        "role": "USER"
      },
      "community": {
        "id": 3,
        "title": "Career Advice",
        "avatar_path": "default_community.png"
      }
    }
  ]
}
```

Example: the users list as an admin (real)

Request

```
GET /api/admin/users?limit=1&sort=login
```

Response


```
HTTP 200
{
  "meta": {
    "total": 9,
    "page": 1,
    "limit": 1,
    "totalPages": 9
  },
  "data": [
    {
      "id": 1,
      "login": "admin",
      "full_name": "Site Administrator",
      "avatar_path": "default_avatar.png",
      "rating": 9,
      "role": "ADMIN",
      "created_at": "2026-10-11T19:18:59.437Z",
      "email": "admin@example.com",
      "is_email_confirmed": true,
      "is_suspended": false,
      "suspended_reason": null,
      "suspended_at": null,
      "_count": {
        "posts": 2,
        "comments": 2,
        "reports_against": 0
      }
    }
  ]
}
```

Example: the same list as a site moderator, no emails (real)

Request

```
GET /api/admin/users?limit=1&sort=login
```

Response

```
HTTP 200
{
  "meta": {
    "total": 9,
    "page": 1,
    "limit": 1,
    "totalPages": 9
  },
  "data": [
    {
      "id": 1,
      "login": "admin",
      "full_name": "Site Administrator",
      "avatar_path": "default_avatar.png",
      "rating": 9,
      "role": "ADMIN",
      "created_at": "2026-10-11T19:18:59.437Z",
      "is_suspended": false,
      "suspended_reason": null,
      "suspended_at": null,
      "_count": {
        "posts": 2,
        "comments": 2,
        "reports_against": 0
      }
    }
  ]
}
```

6. Main objects

The fields below are taken from real responses. Values are shortened.

6.1 Post


```
{
  "id": 2,
  "title": "Promise.all vs allSettled",
  "content": "Detailed question or discussion body for \"Promise.all vs allSett...",
  "publish_date": "2026-10-11T19:18:59.487Z",
  "status": "ACTIVE",
  "score": 1,
  "author_id": 1,
  "community_id": 1,
  "author": {
    "id": 1,
    "login": "admin",
    "full_name": "Site Administrator",
    "avatar_path": "default_avatar.png",
    "rating": 8
  },
  "community": {
    "id": 1,
    "title": "JavaScript Corner",
    "avatar_path": "default_community.png"
  },
  "categories": [
    {
      "id": 3,
      "title": "Showcase",
      "description": null,
      "is_default": false,
      "community_id": 1
    }
  ],
  "media": [
    {
      "id": 2,
      "media_url": "/storage/seed-2.png",
      "type": "IMAGE",
      "post_id": 2
    }
  ],
  "comments_count": 2,
  "viewer_reaction": null
}
```


- **score** is likes minus dislikes. **comments_count** counts active comments. **viewer_reaction** is the signed-in viewer's own reaction (LIKE, DISLIKE or null).
- **categories** is the list of local categories; **media** lists images and videos with their URLs.

6.2 Community


```
{
  "id": 5,
  "title": "Home Lab",
  "description": "Servers, networking and self-hosting",
  "avatar_path": "default_community.png",
  "created_at": "2026-10-11T19:18:59.466Z",
  "owner_id": 6,
  "owner": {
    "id": 6,
    "login": "erin",
    "full_name": "Erin Brown",
    "avatar_path": "default_avatar.png",
    "rating": 3
  },
  "global_categories": [
    {
      "id": 4,
      "title": "Hardware",
      "description": "Computers, electronics and gadgets"
    }
  ],
  "_count": {
    "members": 5,
    "posts": 2
  },
  "viewer_role": "MEMBER"
}
```


6.3 User

As seen by an admin; others do not get email, is_email_confirmed and the counts


```
{
  "id": 1,
  "login": "admin",
  "full_name": "Site Administrator",
  "avatar_path": "default_avatar.png",
  "rating": 9,
  "role": "ADMIN",
  "created_at": "2026-10-11T19:18:59.437Z",
  "email": "admin@example.com",
  "is_email_confirmed": true,
  "is_suspended": false,
  "suspended_reason": null,
  "suspended_at": null,
  "_count": {
    "posts": 2,
    "comments": 2,
    "reports_against": 0
  }
}
```

6.4 Report


```
{
  "id": 7,
  "target_type": "POST",
  "reason": "SPAM",
  "details": "Looks like advertising",
  "target_excerpt": "Promise.all vs allSettled",
  "status": "OPEN",
  "action_taken": null,
  "resolution_note": null,
  "created_at": "2026-10-11T19:19:07.316Z",
  "resolved_at": null,
  "reporter_id": 6,
  "community_id": 1,
  "post_id": 2,
  "comment_id": null,
  "target_user_id": null,
  "resolved_by_id": null,
  "reporter": {
    "id": 6,
    "login": "erin",
    "full_name": "Erin Brown"
  },
  "community": {
    "id": 1,
    "title": "JavaScript Corner",
    "avatar_path": "default_community.png"
  },
  "post": {
    "id": 2,
    "title": "Promise.all vs allSettled",
    "status": "ACTIVE",
    "author_id": 1
  },
  "comment": null,
  "target_user": null,
  "resolved_by": null
}
```


- **target_excerpt** keeps a short copy of the reported text: a report outlives the content that was deleted because of it (**post** or **comment** becomes null).
- **action_taken** after handling: NONE, CONTENT_HIDDEN, CONTENT_DELETED, USER_BANNED, USER_SUSPENDED.

7. Rules and permissions in short

Action	Who may do it
Read active content, communities, categories	everyone
Read hidden (INACTIVE) posts and comments	the author, community staff, site staff
Create a community, comment, react, report	any signed-in user (comments and reactions only on active content)
Create a post	a member of the community (or an admin); not a banned user
Edit the text and media of a post or comment	the author only
Hide or restore a post or comment, move a post to other categories	community staff, admins, site moderators
Delete a post or comment	the author, community staff, admins (not site moderators)
Ban a user from a community	community staff, admins
Suspend an account, change roles, see emails	admins only
Manage local categories, edit the community, remove members	community staff (and admins)
Appoint moderators, transfer ownership, set global categories of a community	the owner (and admins)
Manage global categories, decide about global category requests	admins only

Bans and suspensions

	Community ban	Account suspension
Scope	one community	the whole site
Who issues it	community staff, admins	admins
Effect	removed from the community; cannot join, post or comment there	cannot log in; all tokens stop working
Length	permanent or a number of days	until an admin restores the account
Protected	owners, moderators, admins and site moderators	admins

8. Error examples

400: a request without a body

Request

```
POST /api/auth/login
```

Response

```
HTTP 400
{
  "message": "Login is required"
}
```

401: no token

Request

```
GET /api/posts/feed
```

Response

```
HTTP 401
{
  "message": "Access denied. No token provided"
}
```

403: not allowed

Request

```
GET /api/admin/overview
```

Response

```
HTTP 403
{
  "message": "Insufficient permissions"
}
```

404: not found

Request

```
GET /api/communities/9999
```

Response

```
HTTP 404
{
  "message": "Community not found"
}
```

409: conflict

Request

```
POST /api/communities
Content-Type: application/json

{
  "title": "Databases"
}
```

Response

```
HTTP 409
{
  "message": "Community with this title already exists"
}
```

Appendix A. Enumerations

Name	Values
Site role	`USER, MODERATOR, ADMIN`
Community role	`OWNER, MODERATOR, MEMBER`
Post status	`ACTIVE, INACTIVE, DRAFT, ARCHIVED`
Comment status	`ACTIVE, INACTIVE`
Reaction type	`LIKE, DISLIKE`
Media type	`IMAGE, VIDEO`
Report target	`POST, COMMENT, USER`
Report reason	`SPAM, HARASSMENT, HATE_SPEECH, MISINFORMATION, INAPPROPRIATE, OTHER`
Report status	`OPEN, RESOLVED, DISMISSED`
Report action (resolve)	`NONE, HIDE_CONTENT, DELETE_CONTENT, BAN_USER, SUSPEND_USER`
Category request	`kind GLOBAL or LOCAL; status PENDING, APPROVED, REJECTED`

Appendix B. Activity log actions

`post.hidden`, `post.restored`, `post.deleted`, `post.recategorized`, `comment.hidden`, `comment.restored`, `comment.deleted`, `user.banned`, `user.unbanned`, `user.kicked`, `user.suspended`, `user.unsuspended`, `member.role_changed`, `community.ownership_transferred`, `category.created`, `category.updated`, `category.deleted`, `category_request.approved`, `category_request.rejected`, `report.resolved`, `report.dismissed`

What an author does to their own content is not logged. The log is an activity feed for moderators, not a security audit trail.